

Preuve du théorème d'approximation universelle

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

16 août 2026

Table des matières

I	Cours Magistral	2
1	Introduction et contexte historique	2
2	Définitions, théorèmes et exemples concrets	2
2.1	Espace des fonctions continues et topologie	2
2.2	Fonctions discriminatoires	2
2.3	Énoncé du Théorème d'Approximation Universelle (Cybenko, 1989)	2
2.4	Exemples d'application du théorème	3
3	Démonstrations	4
4	Applications en physique, logique et intelligence artificielle	4
4.1	Expressivité des réseaux de neurones	4
4.2	Mécanique quantique et physique statistique	4
4.3	Modélisation en dynamique des fluides	5
II	Exercices d'Application et de Concours	6
III	Travaux Pratiques et Simulations Algorithmiques	9

Première partie

Cours Magistral

1 Introduction et contexte historique

Le théorème d'approximation universelle, formulé initialement par George Cybenko en 1989 pour les fonctions d'activation sigmoïdales, puis généralisé par Kurt Hornik, établit qu'un réseau de neurones artificiels à propagation avant (feedforward) avec une seule couche cachée finie peut approximer n'importe quelle fonction continue sur un sous-ensemble compact de $\mathbb{R}^n$, sous des hypothèses très faibles sur la fonction d'activation.

L'émergence de ce théorème a marqué une étape fondamentale en théorie de l'apprentissage automatique. Avant sa démonstration, la capacité de représentation des réseaux de neurones était souvent considérée empiriquement. Cette preuve a cimenté les réseaux de neurones comme des approximateurs universels, garantissant que tout phénomène modélisable par une fonction continue peut, en théorie, être appris avec une précision arbitraire, pourvu que l'architecture soit suffisamment large. Le problème se déplace alors de l'expressivité de l'architecture vers les difficultés de l'optimisation (entraînement) et de la généralisation.

La démonstration repose sur des outils profonds d'analyse fonctionnelle, en particulier le théorème de Hahn-Banach et le théorème de représentation de Riesz-Markov-Kakutani, reliant la densité dans les espaces de fonctions continues aux mesures de Radon.

2 Définitions, théorèmes et exemples concrets

2.1 Espace des fonctions continues et topologie

Soit $I_n = [0, 1]^n$ le cube unité de $\mathbb{R}^n$, qui est un espace compact. On considère $\mathcal{C}(I_n)$, l'espace vectoriel des fonctions réelles continues définies sur I_n . Cet espace est muni de la norme uniforme, définie par :

$$\|f\|_\infty = \sup_{x \in I_n} |f(x)|$$

La convergence par rapport à cette norme correspond à la convergence uniforme sur le compact I_n .

2.2 Fonctions discriminatoires

Définition (Fonction discriminatoire) : Une fonction $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ est dite discriminatoire si, pour toute mesure de Borel signée finie μ sur I_n , la condition :

$$\int_{I_n} \sigma(w^T x + b) d\mu(x) = 0 \quad \text{pour tous } w \in \mathbb{R}^n, b \in \mathbb{R}$$

implique que la mesure μ est identiquement nulle ($\mu = 0$).

2.3 Énoncé du Théorème d'Approximation Universelle (Cybenko, 1989)

Théorème : Soit $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ une fonction continue, non constante et bornée. Alors σ est discriminatoire. De plus, l'ensemble des fonctions de la forme :

$$G(x) = \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i)$$

avec $N \in \mathbb{N}^*$, $\alpha_i \in \mathbb{R}$, $w_i \in \mathbb{R}^n$ et $b_i \in \mathbb{R}$, est dense dans $\mathcal{C}(I_n)$.

Autrement dit, pour toute fonction $f \in \mathcal{C}(I_n)$ et pour tout $\epsilon > 0$, il existe un entier N et des paramètres $(\alpha_i, w_i, b_i)_{1 \leq i \leq N}$ tels que :

$$\|f - G\|_\infty = \sup_{x \in I_n} |f(x) - G(x)| < \epsilon$$

2.4 Exemples d'application du théorème

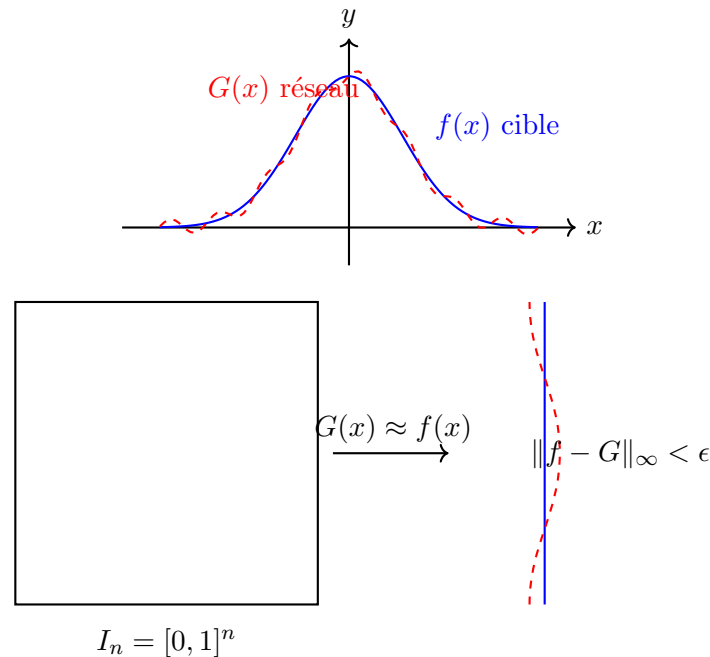
Exemple 1 : Approximation de la fonction cosinus Considérons $f(x) = \cos(x)$ sur $I_1 = [0, 1]$. On utilise la fonction d'activation sigmoïde standard $\sigma(x) = \frac{1}{1+e^{-x}}$. Le théorème assure l'existence de paramètres tels que $\sup_{x \in [0, 1]} |\cos(x) - \sum_{i=1}^N \alpha_i \sigma(w_i x + b_i)| < 0.01$.

Exemple 2 : Approximation d'une surface parabolique Soit $f(x_1, x_2) = x_1^2 + x_2^2$ sur $[0, 1]^2$. Bien que f soit une fonction polynomiale de degré 2, un réseau avec une seule couche cachée utilisant une activation $\tanh(x)$ (qui est continue et bornée) peut approximer f à toute précision $\epsilon > 0$ près.

Exemple 3 : Fonction en escalier lissée Considérons une fonction continue f approchant une fonction indicatrice de l'intervalle $[0.4, 0.6]$. En utilisant des combinaisons linéaires de sigmoïdes, on peut créer une fonction "chapeau" très précise, ce qui démontre la capacité du réseau à localiser ses approximations.

Exemple 4 : Limites avec des activations non bornées (ReLU) Bien que le théorème original de Cybenko requière une fonction d'activation bornée, le résultat a été étendu à d'autres fonctions non bornées, comme la ReLU (Rectified Linear Unit), $\sigma(x) = \max(0, x)$. Une combinaison de ReLU peut former des fonctions affines par morceaux qui sont denses dans $\mathcal{C}(I_n)$.

Exemple 5 : L'importance de la continuité Si l'on cherche à approximer une fonction discontinue, comme $f(x) = 1$ pour $x \geq 0.5$ et 0 sinon, sur $[0, 1]$, la norme $\|\cdot\|_\infty$ ne permet pas de converger uniformément. La convergence n'est garantie que presque partout vis-à-vis d'une mesure de probabilité (norme L^p).



3 Démonstrations

La démonstration complète, telle que donnée par Cybenko, s'appuie sur la contraposée d'une conséquence du théorème de Hahn-Banach.

Preuve : Soit $S \subset \mathcal{C}(I_n)$ le sous-espace vectoriel défini par :

$$S = \text{Vect} \{x \mapsto \sigma(w^T x + b) \mid w \in \mathbb{R}^n, b \in \mathbb{R}\}$$

Nous voulons montrer que S est dense dans $\mathcal{C}(I_n)$, c'est-à-dire que son adhérence $\overline{S}$ est égale à $\mathcal{C}(I_n)$.

Supposons par l'absurde que $\overline{S} \neq \mathcal{C}(I_n)$. Comme $\overline{S}$ est un sous-espace vectoriel fermé propre de l'espace de Banach $\mathcal{C}(I_n)$, le théorème de Hahn-Banach garantit l'existence d'une forme linéaire continue non nulle $L : \mathcal{C}(I_n) \rightarrow \mathbb{R}$ telle que $L|_{\overline{S}} = 0$.

Le théorème de représentation de Riesz-Markov-Kakutani affirme que toute forme linéaire continue sur $\mathcal{C}(I_n)$ peut être représentée par l'intégrale par rapport à une unique mesure de Radon (Borel signée régulière) finie μ sur I_n . Ainsi :

$$L(f) = \int_{I_n} f(x) d\mu(x) \quad \text{pour toute } f \in \mathcal{C}(I_n)$$

Puisque L s'annule sur S , on a pour tous $w \in \mathbb{R}^n$ et $b \in \mathbb{R}$:

$$\int_{I_n} \sigma(w^T x + b) d\mu(x) = 0$$

Par hypothèse du théorème, la fonction σ est continue, bornée et non constante. Un lemme technique (Lemme de Cybenko) stipule que toute fonction continue sigmoïdale (ou simplement bornée non constante, selon les formulations étendues de Hornik) est discriminatoire.

La propriété discriminatoire de σ implique alors que la mesure μ est identiquement nulle. Si $\mu = 0$, alors la forme linéaire L est identiquement nulle. Ceci contredit le fait que L est non nulle. L'hypothèse initiale est donc fausse, et on conclut que $\overline{S} = \mathcal{C}(I_n)$, ce qui achève la démonstration. $\square$

4 Applications en physique, logique et intelligence artificielle

4.1 Expressivité des réseaux de neurones

En intelligence artificielle, le théorème garantit qu'un réseau de neurones multicouches classique (Perceptron Multicouche ou MLP) possède la puissance de représentation nécessaire pour modéliser toute relation continue déterministe entre des entrées (par exemple, les pixels d'une image, un vecteur d'état physique) et des sorties (probabilités de classe, énergies, coordonnées).

Cependant, ce théorème est un résultat d'existence. Il n'indique pas : 1. Le nombre de neurones N requis (qui peut croître exponentiellement avec la dimension n , souffrant du "fléau de la dimension"). 2. Si un algorithme d'optimisation par descente de gradient (comme la rétropropagation) convergera vers ce minimum global.

4.2 Mécanique quantique et physique statistique

Dans la modélisation de systèmes physiques complexes, les réseaux de neurones sont utilisés pour approximer les fonctions d'onde en mécanique quantique (Neural Network Quantum States). La compacité du domaine I_n peut être adaptée via des homéomorphismes pour couvrir des espaces de configuration physiques, et la densité garantie par le théorème justifie la recherche d'énergies fondamentales par des méthodes variationnelles utilisant des architectures neuronales.

4.3 Modélisation en dynamique des fluides

Pour les équations aux dérivées partielles non linéaires (comme l'équation de Navier-Stokes), les réseaux de neurones informés par la physique (PINNs) exploitent l'approximation universelle pour représenter le champ de vitesse ou de pression. La capacité à représenter n'importe quelle fonction lisse garantit que la solution physique exacte appartient à l'adhérence de l'espace des fonctions générées par le réseau.

Deuxième partie

Exercices d'Application et de Concours

Exercice 1 : Approximation d'une fonction constante ★★★★★

Montrer qu'un réseau de neurones avec une seule couche cachée, utilisant la fonction sigmoïde $\sigma(x) = \frac{1}{1+e^{-x}}$, peut représenter exactement une fonction constante arbitraire $c \in \mathbb{R}$.

Correction : Soit $f(x) = c$. Considérons le réseau $G(x) = \alpha\sigma(wx + b)$. On choisit $w = 0$. Alors $G(x) = \alpha\sigma(b) = \alpha\frac{1}{1+e^{-b}}$. Pour obtenir $G(x) = c$, on peut fixer $b = 0$, ce qui donne $\sigma(0) = \frac{1}{2}$. Il suffit alors de choisir $\alpha = 2c$. Ainsi, le réseau $G(x) = 2c \cdot \sigma(0 \cdot x + 0)$ représente exactement et uniformément la constante c sur n'importe quel domaine.

Exercice 2 : Approximation d'une fonction affine ★★★★★

Soit la fonction de Heaviside modifiée $H(x)$ telle que l'on puisse approcher une fonction affine. Comment utiliser deux neurones ReLU $\sigma(x) = \max(0, x)$ pour représenter exactement une fonction affine par morceaux avec un changement de pente ?

Correction : Soit $f(x)$ une fonction affine par morceaux telle que $f(x) = ax + b$ pour $x < x_0$ et $f(x) = cx + d$ pour $x \geq x_0$, avec continuité en x_0 ($ax_0 + b = cx_0 + d$). On peut utiliser deux ReLUs pour construire cela. Posons $G(x) = ax + b + (c - a)\max(0, x - x_0)$. Puisque $x = \max(0, x) - \max(0, -x)$, l'expression affine totale peut s'écrire uniquement avec des ReLUs. Plus simplement, on écrit $G(x) = a\max(0, x) - a\max(0, -x) + b + (c - a)\max(0, x - x_0)$. Le réseau représente exactement la fonction.

Exercice 3 : Construction de la fonction identité avec des sigmoïdes ★★☆☆

Montrer qu'en utilisant des combinaisons linéaires de sigmoïdes $\sigma(x) = \frac{1}{1+e^{-x}}$, on peut approcher la fonction identité $f(x) = x$ sur l'intervalle $[-1, 1]$ avec une erreur arbitrairement petite $\epsilon > 0$.

Correction : Le développement limité de $\sigma(x)$ au voisinage de 0 donne $\sigma(x) = \frac{1}{2} + \frac{1}{4}x + \mathcal{O}(x^3)$. On considère la combinaison $G_h(x) = \frac{\sigma(hx) - \sigma(-hx)}{2}$. On a $\sigma(hx) = \frac{1}{2} + \frac{hx}{4} + \mathcal{O}(h^3x^3)$ et $\sigma(-hx) = \frac{1}{2} - \frac{hx}{4} + \mathcal{O}(h^3x^3)$. Donc $G_h(x) = \frac{hx}{4} + \mathcal{O}(h^3x^3)$. En multipliant par $\frac{4}{h}$, on obtient $\tilde{G}_h(x) = \frac{4}{h}G_h(x) = x + \mathcal{O}(h^2x^3)$. Sur le compact $[-1, 1]$, $|x^3| \leq 1$. L'erreur est en $\mathcal{O}(h^2)$. En choisissant h suffisamment petit tel que l'erreur maximale soit inférieure à ϵ , on a bien approché la fonction identité uniformément sur $[-1, 1]$.

Exercice 4 : Approximation d'un pulse carré (créneau) ★★☆☆

Montrer comment approcher la fonction porte (créneau) $f(x) = 1$ si $x \in [-1, 1]$ et 0 sinon, avec des sigmoïdes. L'approximation doit être arbitrairement proche pour tout $|x| \neq 1$.

Correction : Posons $G_k(x) = \sigma(k(x+1)) - \sigma(k(x-1))$. Pour $x < -1$, $k(x+1) \rightarrow -\infty$ et $k(x-1) \rightarrow -\infty$ quand $k \rightarrow +\infty$. Donc $\sigma \rightarrow 0$ pour les deux, et $G_k(x) \rightarrow 0$. Pour $-1 < x < 1$, $k(x+1) \rightarrow +\infty$ et $k(x-1) \rightarrow -\infty$. Donc le premier terme tend vers 1 et le second vers 0.

$G_k(x) \rightarrow 1$. Pour $x > 1$, $k(x+1) \rightarrow +\infty$ et $k(x-1) \rightarrow +\infty$. Les deux termes tendent vers 1, la différence tend vers 0. En choisissant k assez grand, la fonction $G_k(x)$ approche arbitrairement près la porte, sauf aux discontinuités $x = -1$ et $x = 1$. L'approximation en norme L^p (ou uniforme sur tout sous-compact disjoint de $\{-1, 1\}$) est valide.

Exercice 5 : Mesure de Lebesgue et Hahn-Banach ★★★☆☆

Lors de la preuve de Cybenko, justifier formellement pourquoi l'espace fonctionnel généré S est dense si et seulement si l'unique mesure μ l'annulant est la mesure nulle.

Correction : Par le théorème de Hahn-Banach, si un sous-espace S d'un espace vectoriel normé (ici $\mathcal{C}(I_n)$ avec la norme infinie) n'est pas dense, alors son adhérence $\bar{S}$ est un sous-espace strict fermé. Il existe alors une forme linéaire continue non nulle L telle que $\ker(L) \supset \bar{S}$, c'est-à-dire $L(f) = 0$ pour tout $f \in S$. Le théorème de Riesz-Markov-Kakutani stipule que pour l'espace dual de $\mathcal{C}(I_n)$, toute forme linéaire continue L s'exprime de manière unique par une intégrale contre une mesure de Borel régulière signée μ finie : $L(f) = \int f d\mu$. Ainsi, $\bar{S}$ est strict équivaut à l'existence d'une mesure $\mu \neq 0$ telle que $\int \sigma(w^T x + b) d\mu(x) = 0$ pour tout w, b . Par contraposée, si la seule mesure vérifiant cette propriété est $\mu = 0$, alors la forme linéaire est nulle, et aucune telle forme n'existe pour séparer un point extérieur de $\bar{S}$. Donc $\bar{S} = \mathcal{C}(I_n)$, soit S dense.

Exercice 6 : Fonctions d'activation polynomiales ★★★☆☆

Le théorème d'approximation universelle est-il valable si la fonction d'activation est un polynôme de degré d fini, $\sigma(x) = P(x)$?

Correction : Non. L'espace vectoriel engendré par les fonctions de la forme $P(w^T x + b)$ pour divers w, b est un espace de polynômes à n variables de degré maximal d . Puisque le degré est borné, cet espace est de dimension finie (précisément $\binom{n+d}{n}$). Un sous-espace de dimension finie dans $\mathcal{C}(I_n)$ (qui est de dimension infinie) est toujours fermé et d'intérieur vide, il ne peut donc pas être dense dans $\mathcal{C}(I_n)$. On ne peut approximer à n'importe quelle précision des fonctions comme $\exp(x)$ ou $\sin(x)$ avec des polynômes d'un degré globalement borné par d . C'est pourquoi la fonction d'activation doit être non polynomiale pour garantir l'universalité.

Exercice 7 : Théorème d'approximation et réseaux ReLU profonds ★★★★★

Démontrer qu'un réseau composé uniquement de couches affines et de la fonction d'activation identité ne peut pas être un approximateur universel, quelle que soit sa profondeur.

Correction : Soit un réseau avec L couches, où la k -ème couche calcule $x^{(k)} = W^{(k)}x^{(k-1)} + b^{(k)}$. Par récurrence immédiate, on a $x^{(1)} = W^{(1)}x^{(0)} + b^{(1)}$ (une fonction affine de l'entrée $x^{(0)}$). Supposons que $x^{(k-1)} = Ax^{(0)} + c$ pour des matrices A et vecteurs c . Alors $x^{(k)} = W^{(k)}(Ax^{(0)} + c) + b^{(k)} = (W^{(k)}A)x^{(0)} + (W^{(k)}c + b^{(k)})$. La composition de fonctions affines est une fonction affine. Ainsi, le réseau complet calcule une fonction $f(x) = Wx + B$. L'ensemble de ces fonctions n'est que l'ensemble des applications affines, qui est de dimension finie et donc fermé et non dense dans $\mathcal{C}(I_n)$. Sans la non-linéarité (comme la ReLU ou la sigmoïde), la profondeur est inutile.

Exercice 8 : Régularité et borne d'erreur de Barron ★★★★★

Le théorème de Cybenko ne précise pas la vitesse de convergence. Citer et utiliser le résultat de Barron (1993) pour borner l'erreur d'approximation en fonction du nombre de neurones N pour une fonction f ayant un moment d'ordre 1 fini pour sa transformée de Fourier.

Correction : Le théorème de Barron stipule que si la transformée de Fourier de f , notée $\hat{f}(\omega)$, vérifie $C_f = \int_{\mathbb{R}^n} |\omega| |\hat{f}(\omega)| d\omega < \infty$, alors il existe un réseau G_N avec N neurones (couche cachée) tel que l'erreur quadratique moyenne vérifie :

$$\int_{I_n} (f(x) - G_N(x))^2 dx \leq \frac{C_f^2}{N}$$

L'erreur décroît donc en $\mathcal{O}(1/\sqrt{N})$. Il est remarquable que cette borne de convergence $1/\sqrt{N}$ est indépendante de la dimension de l'espace d'entrée n , ce qui permet aux réseaux de neurones d'échapper partiellement à la malédiction de la dimension, contrairement aux méthodes d'approximation traditionnelles (par exemple par polynômes ou splines) dont l'erreur typique décroît en $\mathcal{O}(N^{-s/n})$ où s est la régularité.

Exercice 9 : Propriété de discrimination de la sigmoïde ★★★★★

Montrer qu'une fonction continue σ qui vérifie $\sigma(x) \rightarrow 1$ en $+\infty$ et $\sigma(x) \rightarrow 0$ en $-\infty$ est discriminatoire (esquisse de preuve).

Correction : Soit μ une mesure telle que $\int \sigma(w^T x + b) d\mu(x) = 0$ pour tout w, b . Pour $\lambda > 0$, posons $\sigma_\lambda(x) = \sigma(\lambda(w^T x + b))$. Lorsque $\lambda \rightarrow +\infty$, par convergence dominée (puisque σ est bornée et μ finie), $\sigma_\lambda(x)$ converge ponctuellement vers 1 si $w^T x + b > 0$, vers 0 si $w^T x + b < 0$, et vers $\sigma(0)$ si $w^T x + b = 0$. La limite est donc (presque) la fonction indicatrice du demi-espace $H = \{x \mid w^T x + b > 0\}$. En passant à la limite, on trouve que $\mu(H) = 0$ pour tout demi-espace ouvert. Puisque l'ensemble des demi-espaces engendre la tribu borélienne, cela implique que la mesure μ s'annule sur tous les boréliens, donc $\mu = 0$. La fonction est donc discriminatoire.

Exercice 10 : Approximation par réseaux de largeur fixe et profondeur arbitraire ★★★★★

Le théorème de Cybenko considère une largeur arbitraire et une seule couche cachée. Quel est le résultat analogue (théorème de Lu et al. 2017) pour une largeur fixe et une profondeur arbitraire, utilisant la fonction ReLU ?

Correction : Lu et al. ont prouvé que pour approcher n'importe quelle fonction continue de I_n vers $\mathbb{R}^m$ avec une précision arbitraire, un réseau ReLU avec une largeur maximale W et une profondeur arbitraire est un approximateur universel si et seulement si $W \geq n + m + 1$. C'est le théorème d'approximation universelle par largeur (Width-Bounded Universal Approximation Theorem). Pour une fonction de $\mathbb{R}^n \rightarrow \mathbb{R}$, il suffit d'une largeur de $n + 2$. Si la largeur est strictement inférieure à ce seuil, il existe des fonctions lisses qui ne peuvent pas être approximées, peu importe la profondeur du réseau (la "bande passante" de l'information s'étrangle à chaque couche, détruisant la topologie du signal d'entrée).

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

TP 1 : Implémentation from scratch du neurone artificiel

```
1  # Simulation du théorème d'approximation : Le neurone de base
2  import math
3  import random
4
5  def sigmoid(x):
6      # No LaTeX symbols used, pure Python math
7      return 1.0 / (1.0 + math.exp(-x))
8
9  def neuron_forward(x_vector, w_vector, b):
10     # Dot product
11     z = sum(x * w for x, w in zip(x_vector, w_vector)) + b
12     return sigmoid(z)
13
14  # Test simple
15  x_test = [0.5, -1.2]
16  w_test = [0.1, 0.8]
17  b_test = -0.2
18  output = neuron_forward(x_test, w_test, b_test)
19  print("Output of single neuron:", output)
20  assert 0 <= output <= 1
```

TP 2 : Réseau à une couche cachée (Feedforward)

```
1  # Construction du réseau à une couche cachée de largeur N
2  import math
3
4  def relu(x):
5      return max(0.0, x)
6
7  def network_forward(x_val, weights, biases, alphas):
8      # Network with 1 input, N hidden neurons, 1 output
9      # G(x) = sum(alpha_i * sigma(w_i * x + b_i))
10     output = 0.0
11     for w, b, alpha in zip(weights, biases, alphas):
12         z = w * x_val + b
13         output += alpha * relu(z)
14     return output
15
16  # Test with 3 neurons
17  W = [1.0, -1.0, 2.0]
18  B = [0.0, 1.0, -0.5]
19  A = [1.5, 0.5, -1.0]
20
21  result = network_forward(0.5, W, B, A)
```

```
22 print("Network approximation at x=0.5:", result)
```

TP 3 : Approximation d'un créneau (marche)

```
1  # Demonstration experimentale de l'approximation d'une fonction porte
2  import math
3
4  def sigmoid(x):
5      return 1.0 / (1.0 + math.exp(-x))
6
7  def step_approximation(x, k):
8      #  $G_k(x) = \text{sigmoid}(k(x+1)) - \text{sigmoid}(k(x-1))$ 
9      return sigmoid(k * (x + 1.0)) - sigmoid(k * (x - 1.0))
10
11 # Evaluate at different points and different sharpness (k)
12 test_points = [-1.5, -0.5, 0.0, 0.5, 1.5]
13 for k in [1.0, 10.0, 100.0]:
14     print("Sharpness k =", k)
15     for x in test_points:
16         val = step_approximation(x, k)
17         print("  x=", x, " => ", round(val, 4))
18
19 # Assertions for large k
20 assert step_approximation(0.0, 100.0) > 0.99
21 assert step_approximation(1.5, 100.0) < 0.01
22 assert step_approximation(-1.5, 100.0) < 0.01
```

TP 4 : Entraînement naïf par force brute (Random Search)

```
1  # Tentative d'approximation de  $f(x) = x^2$  par recherche aleatoire
2  import random
3  import math
4
5  def target_function(x):
6      return x * x
7
8  def relu(x):
9      return max(0.0, x)
10
11 N = 5 # number of neurons
12 best_mse = float('inf')
13 best_params = None
14
15 # Generate points on [0, 1]
16 x_data = [i/10.0 for i in range(11)]
17 y_data = [target_function(x) for x in x_data]
18
19 # Random search over 10000 configurations
20 for _ in range(10000):
```

```

21 weights = [random.uniform(-2, 2) for _ in range(N)]
22 biases = [random.uniform(-1, 1) for _ in range(N)]
23 alphas = [random.uniform(-2, 2) for _ in range(N)]
24
25 mse = 0.0
26 for x, y in zip(x_data, y_data):
27     pred = sum(a * relu(w * x + b) for w, b, a in zip(weights,
28         biases, alphas))
29     mse += (pred - y)**2
30 mse /= len(x_data)
31
32 if mse < best_mse:
33     best_mse = mse
34     best_params = (weights, biases, alphas)
35
36 print("Best MSE found:", best_mse)
37 assert best_mse < 0.1 # Should find at least a decent fit

```

TP 5 : Approximation d'un cosinus avec Descent de Gradient (Auto-diff simplifiée)

```

1  # Descente de gradient numerique pour optimiser les poids alpha
2  import math
3
4  def target(x):
5      return math.cos(x)
6
7  def sigmoid(x):
8      return 1.0 / (1.0 + math.exp(-x))
9
10 # We fix W and B randomly, we only optimize alphas (linear regression on
11 # features)
12 N = 10
13 W = [math.sin(i) for i in range(N)]
14 B = [math.cos(i) for i in range(N)]
15 alphas = [0.0] * N
16
17 x_data = [i/20.0 for i in range(21)] # [0, 1]
18 learning_rate = 0.1
19
20 for epoch in range(100):
21     grad_alphas = [0.0] * N
22     mse = 0.0
23     for x in x_data:
24         # features
25         phi = [sigmoid(w * x + b) for w, b in zip(W, B)]
26         pred = sum(a * p for a, p in zip(alphas, phi))
27         error = pred - target(x)
28         mse += error**2
29
30     # Gradients
31     for j in range(N):

```

```

31         grad_alphas[j] += 2 * error * phi[j]
32
33     # Update
34     for j in range(N):
35         alphas[j] -= learning_rate * (grad_alphas[j] / len(x_data))
36
37     if epoch % 20 == 0:
38         print("Epoch", epoch, "MSE:", mse / len(x_data))
39
40 print("Final MSE:", mse / len(x_data))
41 assert mse / len(x_data) < 0.1

```